

Clase 6

Análisis y eficiencia de algoritmos

Hay que definir cómo evaluamos la eficiencia de los algoritmos:

- Criterio temporal → cuánto tarda en ejecutarse.
- Criterio espacial → cuánto consume de recursos físicos.

El análisis temporal, a su vez, se divide en:

- Análisis empírico: ejecutar y ver cuánto tarda. Se plantean distintos casos de prueba varias veces, ya que puede haber algunas fuentes de incertidumbre temporal relacionadas al sistema operativo, acceso a periféricos, etcétera, se obtiene el promedio y se plantea un gráfico de dispersión.
Sin embargo, tiene el inconveniente de que depende de la máquina en la que estás ejecutando el código y el lenguaje utilizado.
Se debe indicar en qué circunstancias se desarrolló la prueba.
- Análisis teórico: plantear una fórmula que lo represente. No necesita implementarse y su resultado es aplicable para cualquier máquina y lenguaje.
Trata de encontrar la curva que mejor se ajuste al desempeño en base a la cantidad de datos de entrada.
Las acciones atómicas son aquellas que ocupan un tiempo de máquina.
Por ejemplo, todas las acciones que se pueden realizar en Assembler. El tiempo variará según la computadora, pero sigue siendo un tiempo de máquina.
No importa saber cómo se comportan los algoritmos para una cantidad específica de n sino con una tendencia a infinito.

En esta materia nos centramos más en el análisis teórico.

- Operaciones atómicas: ocupan un tiempo de reloj. Por ejemplo, asignar valor a una variable, devolver un valor en una función, evaluar una expresión lógica, etcétera.
- Operaciones de bucles: se multiplica lo que tarda el algoritmo por la cantidad de iteraciones. El estudio que se hace es exactamente igual con while que con for. Si hay bucles anidados con cantidad de iteraciones distintas, se toma el máximo.
- Operaciones con condicionales: como no se entra a todas las opciones, solo a una, se toma la eficiencia del peor caso.

Se calcula el límite n tendiendo a $+\infty$ dividiendo ambas fórmulas de las funciones para saber cuál es más eficiente:

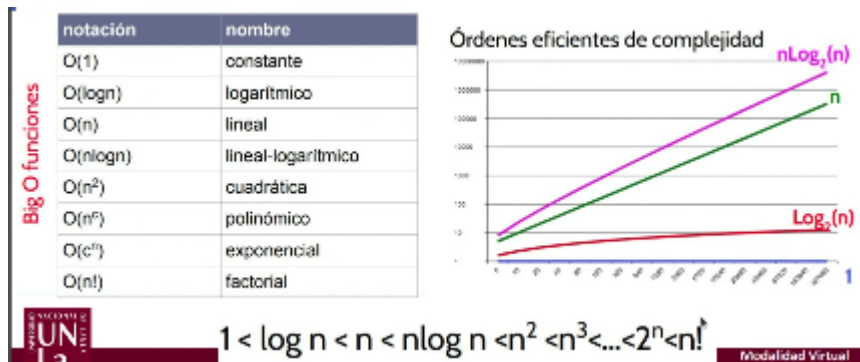
- Si tiende a 1 es que tienen una eficiencia equivalente.
- Si tiende a 0 es porque el algoritmo divisor es menos eficiente.

- Si tiende a + infinito es porque el algoritmo dividiendo es menos eficiente.

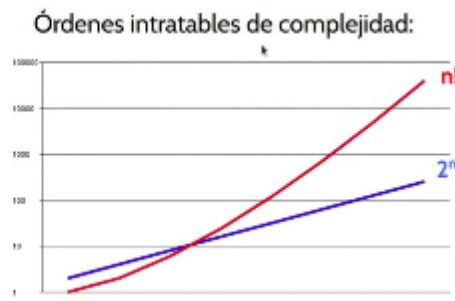
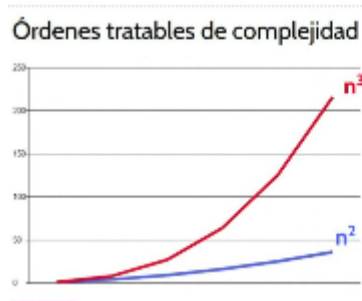
La notación Big O consiste en encontrar el término que más aporta a la eficiencia. Por ejemplo:

T(n)	BigO	T(n)	Big-O
4	O(1)	n + 2	O(n)
3n+4	O(n)	$\frac{1}{2}(n+1)(n-1)$	O(n ²)
5n ² + 27n + 1005	O(n ²)	7n ⁴ +5n ² +1	O(n ⁴)
10n ³ + 2n ² + 7n + 1	O(n ³)	n(n-1)	O(n ²)
nl+ n ^{1/2}	O(nl)	3n+log(n)	O(n)

Órdenes de notación Big O posibles



Si un algoritmo tiene algunas de las eficiencias de las del gráfico, es eficientemente aceptable.



Las tratables son malas, pero las intratables son muy malas. Los primeros pueden ejecutarse a pesar de todo, pero los últimos no y hay que buscar mejorarlos.